Lab 6: Code and Branch Coverage Analysis

Software Engineering Lab — Submission Report

Name	SHRIRAM CHANDRASEKAR
SRN	PES2UG24CS495
Assigned Repository	L6_L7_55_car_insurance_premium
Application	Car Insurance Premium Calculator
Date	29 October 2026

1. Objective

The goal of this lab is to understand the difference between code (line) coverage and branch coverage, use Python's coverage tool to measure and analyse test coverage, interpret the reports to find untested paths, and write additional test cases to achieve at least 90% code coverage and at least 90% branch coverage for the assigned module.

2. Application Under Test

The module car_insurance_premium.py computes yearly car insurance premiums based on car type, driver age and claim history, looks up a no-claim bonus, and calculates claim payouts. It contains three functions with several decision branches each.

car_insurance_premium.py

```
BASE_PREMIUM = {"hatchback": 8000, "sedan": 12000, "suv": 15000}


def annual_premium(car_type, driver_age, claims_last_3_years, has_anti_theft):
    if car_type not in BASE_PREMIUM:
        raise ValueError("unknown car type")
    if driver_age < 18:
        raise ValueError("driver too young")
    premium = BASE_PREMIUM[car_type]
    if driver_age < 25:
        premium += 4000
    elif driver_age >= 60:
        premium += 2000
    if claims_last_3_years == 0:
        premium -= premium // 5
    elif claims_last_3_years > 2:
        premium = premium * 2
    if has_anti_theft:
        premium -= 500
    return premium
```

```
def no_claim_bonus(claim_free_years):
    if claim_free_years < 0:
        raise ValueError("invalid years")
    if claim_free_years >= 5:
        return 50
    elif claim_free_years >= 3:
        return 35
    elif claim_free_years >= 1:
        return 20
    return 0


def claim_payout(damage_cost, insured_value, deductible, is_total_loss):
    if damage_cost < 0 or deductible < 0:
        raise ValueError("invalid amount")
    if is_total_loss:
        return insured_value
    if damage_cost <= deductible:
        return 0
    payout = damage_cost - deductible
    if payout > insured_value:
        return insured_value
    return payout
```

3. Tools and Commands Used

Dependencies were installed with:

```
pip install coverage pytest
```

Code (line) coverage:

```
coverage run -m pytest CodeCoverageTest_PES2UG24CS495_car_insurance_premium.py
coverage report -m
```

Branch coverage:

```
coverage run --branch -m pytest
BranchCoverageTest_PES2UG24CS495_car_insurance_premium.py
coverage report -m
```

4. Code (Line) Coverage

Line coverage asks: “Did execution pass through each line at least once?” The test suite below exercises every statement in the module, including the error-raising lines and every discount/surcharge path.

CodeCoverageTest_PES2UG24CS495_car_insurance_premium.py

```python
import pytest
from car_insurance_premium import annual_premium, no_claim_bonus, claim_payout

# Lab 6 - Code (Line) Coverage test cases
# SRN: PES2UG24CS495  Name: SHRIRAM CHANDRASEKAR
# Goal: execute every line of car_insurance_premium.py at least once.


# ---------- annual_premium ----------
def test_sedan_no_claims():
    # base example: sedan, age 30, 0 claims, no anti-theft
    assert annual_premium("sedan", 30, 0, False) == 9600


def test_unknown_car_type_raises():
    # hits the "unknown car type" line
    with pytest.raises(ValueError):
        annual_premium("truck", 30, 0, False)


def test_driver_too_young_raises():
    # hits the "driver too young" line
    with pytest.raises(ValueError):
        annual_premium("sedan", 17, 0, False)


def test_young_driver_surcharge_with_anti_theft():
    # age < 25 surcharge (+4000), claims not 0 / not >2, anti-theft (-500)
    # hatchback 8000 + 4000 = 12000 - 500 = 11500
    assert annual_premium("hatchback", 20, 1, True) == 11500


def test_senior_driver_many_claims():
    # age >= 60 surcharge (+2000), claims > 2 doubles the premium
    # suv 15000 + 2000 = 17000 * 2 = 34000
    assert annual_premium("suv", 65, 3, False) == 34000


# ---------- no_claim_bonus ----------
def test_no_bonus():
    assert no_claim_bonus(0) == 0


def test_bonus_negative_raises():
    with pytest.raises(ValueError):
        no_claim_bonus(-1)


def test_bonus_tiers():
    assert no_claim_bonus(5) == 50   # >= 5
    assert no_claim_bonus(3) == 35   # >= 3
    assert no_claim_bonus(1) == 20   # >= 1
```

```
# ---------- claim_payout ----------
def test_payout_invalid_amount_raises():
    with pytest.raises(ValueError):
        claim_payout(-100, 50000, 1000, False)


def test_payout_total_loss():
    assert claim_payout(10000, 50000, 1000, True) == 50000


def test_payout_within_deductible():
    assert claim_payout(500, 50000, 1000, False) == 0


def test_payout_capped_at_insured_value():
    # damage - deductible exceeds insured value -> capped
    assert claim_payout(80000, 50000, 1000, False) == 50000


def test_payout_normal():
    # damage - deductible, below insured value
    assert claim_payout(20000, 50000, 1000, False) == 19000
```

4.1 Result

All 13 tests pass and line coverage of car_insurance_premium.py is 100% (well above the 90% requirement).

```
PES2UG24CS495  SHRIRAM CHANDRASEKAR  -  Lab 6 Code (Line) Coverage

> coverage run -m pytest CodeCoverageTest_PES2UG24CS495_car_insurance_premium.py
============================================= test session starts =============================================
platform win32 -- Python 3.12.7, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\cshri\AppData\Local\Temp\claude\C--Users-cshri\362814ae-e40a-4378-8c1f-f5c364660a54\scratchpad\L6_L7_55_car_in
urance_premium\LAB6
collected 13 items

CodeCoverageTest_PES2UG24CS495_car_insurance_premium.py .............                                    [100%]

============================================== 13 passed in 0.05s ==============================================

> coverage report -m
Name                                                Stmts   Miss  Cover   Missing
---------------------------------------------------------------------------------
CodeCoverageTest_PES2UG24CS495_car_insurance_premium.py      34      0   100%
car_insurance_premium.py                              39      0   100%
---------------------------------------------------------------------------------
TOTAL                                                 73      0   100%
```

Figure 1: Code (line) coverage report — 100% coverage

5. Branch Coverage

Branch coverage asks: “Did every control structure (if/elif) evaluate to both True and False?” The suite below adds the missing False/alternate paths — for example a known car type, a driver aged 25–59 (neither young nor senior), and claim counts that are neither 0 nor greater than 2.

BranchCoverageTest_PES2UG24CS495_car_insurance_premium.py

```
import pytest
from car_insurance_premium import annual_premium, no_claim_bonus, claim_payout

# Lab 6 - Branch Coverage test cases
# SRN: PES2UG24CS495  Name: SHRIRAM CHANDRASEKAR
# Goal: make every decision (if / elif) evaluate to BOTH True and False.


# ---------- annual_premium ----------
def test_unknown_car_type_true():
    # car_type not in BASE_PREMIUM -> True
    with pytest.raises(ValueError):
        annual_premium("truck", 30, 0, False)


def test_known_car_type_false_and_age_too_young_true():
    # car_type not in BASE_PREMIUM -> False, driver_age < 18 -> True
    with pytest.raises(ValueError):
        annual_premium("sedan", 17, 0, False)


def test_young_driver_age_lt_25_true():
    # driver_age < 18 -> False, driver_age < 25 -> True,
    # claims == 0 -> False, claims > 2 -> False, has_anti_theft -> True
    # hatchback 8000 + 4000 = 12000 - 500 = 11500
    assert annual_premium("hatchback", 20, 1, True) == 11500


def test_senior_driver_age_ge_60_true():
    # driver_age < 25 -> False, driver_age >= 60 -> True,
    # claims == 0 -> False, claims > 2 -> True, has_anti_theft -> False
    # suv 15000 + 2000 = 17000 * 2 = 34000
    assert annual_premium("suv", 65, 3, False) == 34000


def test_middle_age_both_age_branches_false_claims_zero_true():
    # driver_age < 25 -> False, driver_age >= 60 -> False (25..59),
    # claims == 0 -> True, has_anti_theft -> False
    # sedan 12000 - 2400 = 9600
    assert annual_premium("sedan", 30, 0, False) == 9600


def test_middle_age_claims_gt_2_false_but_not_zero():
    # claims == 0 -> False, claims > 2 -> False (1 or 2 claims)
    # sedan 12000, no adjustments, no anti-theft
    assert annual_premium("sedan", 40, 2, False) == 12000


# ---------- no_claim_bonus ----------
def test_bonus_years_negative_true():
    # claim_free_years < 0 -> True
    with pytest.raises(ValueError):
        no_claim_bonus(-1)
```

```
def test_bonus_ge_5_true():
    # < 0 False, >= 5 True
    assert no_claim_bonus(5) == 50


def test_bonus_ge_3_true():
    # >= 5 False, >= 3 True
    assert no_claim_bonus(3) == 35


def test_bonus_ge_1_true():
    # >= 3 False, >= 1 True
    assert no_claim_bonus(1) == 20


def test_bonus_all_false_returns_zero():
    # >= 5 False, >= 3 False, >= 1 False
    assert no_claim_bonus(0) == 0


# ---------- claim_payout ----------
def test_payout_invalid_amount_true():
    # damage_cost < 0 or deductible < 0 -> True
    with pytest.raises(ValueError):
        claim_payout(-100, 50000, 1000, False)


def test_payout_total_loss_true():
    # invalid -> False, is_total_loss -> True
    assert claim_payout(10000, 50000, 1000, True) == 50000


def test_payout_within_deductible_true():
    # is_total_loss -> False, damage_cost <= deductible -> True
    assert claim_payout(500, 50000, 1000, False) == 0


def test_payout_over_insured_value_true():
    # damage_cost <= deductible -> False, payout > insured_value -> True
    assert claim_payout(80000, 50000, 1000, False) == 50000


def test_payout_normal_all_remaining_false():
    # payout > insured_value -> False
    assert claim_payout(20000, 50000, 1000, False) == 19000
```

5.1 Result

All 16 tests pass. Branch coverage is 100% (30 of 30 branches taken), again exceeding the 90% requirement.

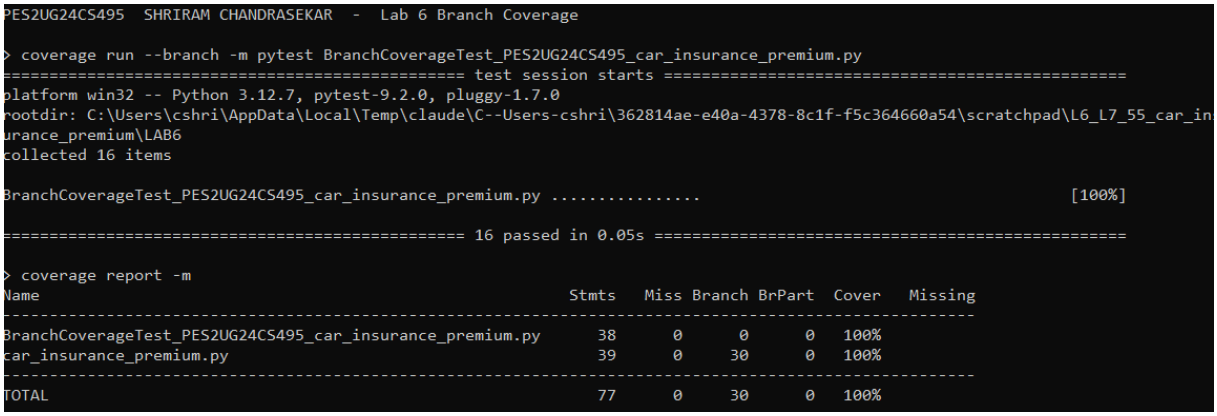


Figure 2: Branch coverage report — 100% coverage (30/30 branches)

6. Summary of Results

Metric	Tests	Coverage	Requirement
Code (line) coverage	13	100%	≥ 90% ✓
Branch coverage	16	100% (30/30)	≥ 90% ✓

7. Deliverables and Commits

The following files were committed to the LAB6 folder of the repository:

- CodeCoverageTest_PES2UG24CS495_car_insurance_premium.py — line coverage tests
- BranchCoverageTest_PES2UG24CS495_car_insurance_premium.py — branch coverage tests
- CodeCoverage_PES2UG24CS495.png — line coverage terminal screenshot
- BranchCoverage_PES2UG24CS495.png — branch coverage terminal screenshot

Two commits were made as required:

1. Commit 1 — Lab 6: code coverage tests + report screenshot (100% line coverage)
2. Commit 2 — Lab 6: branch coverage tests + report screenshot (100% branch coverage)

8. Conclusion

Starting from a two-test starter suite, additional targeted test cases were written to cover every line and every decision branch of the car insurance premium module. Both line coverage and branch coverage reached 100%, comfortably exceeding the 90% target. The exercise demonstrates how line coverage alone can hide untested logical paths, and how branch coverage forces every True/False outcome — and therefore the edge cases — to be tested.